BYTECUBIT TECHNOLOGIES · INDUSTRIAL AUTOMATION ENGINEER

Russell Dudek

Industrial Systems Modernization | Manufacturing Data & Automation

Pittsburgh, Pennsylvania · 412.287.8640
russelldudek@gmail.com · linkedin.com/in/russelldudek
https://russelldudek.github.io/ByteCubit/

90-Day Entry Plan

Stabilize the estate, prove one bounded migration, and leave a reusable release method.

ENTRY PRINCIPLES

Production first
Bound risk, observe behavior, and preserve rollback. A live plant is not a development environment.

Evidence before scale
One well-proven reusable pattern is more valuable than several fast bespoke screens.

Context is data
Names, units, timestamps, quality, asset relationships, ownership, and intended use travel with the signal.

Adoption is engineering
Operator workflow, diagnostics, support, training, and decision rights are part of application design.

DAY-ONE QUESTIONS

- Which production process and function creates the most costly recurring friction?
- What is the Ignition topology, module mix, environment structure, and release path?
- How are tags, UDTs, scripts, queries, projects, and credentials governed?
- Which MES, PI, and SQL dependencies are documented, and which are known only locally?
- Who has authority to accept, hold, cut over, and roll back?
- Which data consumers are most sensitive to naming, timestamp, quality, or asset-model changes?

DAYS 1-30

Read and stabilize

Days 1-15 · Orient and triage

- Walk the process with operators, controls, application, quality, maintenance, R&D, support, and data users.
- Inventory Ignition, legacy MES, SQL, PI, interfaces, environments, recurring issues, and support ownership.
- Map observed states, handoffs, workarounds, consequences, consumers, and diagnosis gaps.

Days 16-30 · Define the method

- Select one bounded pilot with real value, manageable consequence, and representative architecture.
- Establish source control, naming, UDT/tag, component, SQL access, test

DAYS 31-60

Build and reconcile

Days 31-45 · Build the target twin

- Develop the target Ignition component with explicit states, interfaces, diagnostics, and error behavior.
- Map queries, transactions, historian points, contexts, and downstream consumers.
- Test representative events, edge cases, access behavior, and degraded conditions.

Days 46-60 · Reconcile and prepare

- Compare legacy and target behavior, data values, timestamps, quality, and operator task flow.
- Conduct manufacturing, quality, support, R&D, and data-user reviews.
- Close evidence gaps and prepare

Release and Compound

Use the first migration to create reusable application, data, support, and governance patterns.

DAYS 61-75

Release deliberately

- Complete parity, exception, performance, usability, access, and rollback evidence.
- Conduct release-readiness review with named decision owner and production participants.
- Cut over during an agreed production window with communication and support coverage.
- Observe events, data quality, operator use, support signals, and downstream consumers.
- Roll back immediately if pre-agreed thresholds are breached.

DAYS 76-90

Compound the pattern

- Publish reusable Ignition component, tag/UDT, query, diagnostic, test, and release patterns.
- Update PI naming, context, stewardship, and consumer-impact guidance from observed work.
- Define the cloud data-product contract: schema, semantics, quality, freshness, access, owner, and consumer.
- Prioritize the next tranche by value, dependency risk, reuse potential, and readiness.
- Establish a weekly modernization review and monthly architecture/data stewardship review.

90-DAY SCORECARD

Outcome	Evidence at day 90	Operating value
Production value	One bounded function released or a documented hold supported by evidence.	Visible progress without unmanaged production risk.
Modernization method	Signal Twin acceptance record, release/rollback convention, reusable patterns.	Faster and more consistent next-wave delivery.
Data trust	Documented tag, SQL, PI, and cloud context with consumer mapping.	More reliable reporting, analysis, and diagnosis.
Supportability	Diagnostics, runbook, owner, issue path, and post-release observation.	Less person-dependent troubleshooting and handoff risk.
Portfolio direction	Prioritized next tranche with value, risk, dependencies, and standards.	Clear investment and sequencing decisions.

OPERATING CADENCE AND DECISION RIGHTS

Weekly modernization review
Progress, blockers, evidence gaps, production risk, and next decisions.

Monthly architecture and data review
Reusable patterns, historian stewardship, cloud contracts, and technical debt.

Named release authority
Explicit owners for behavior acceptance, data acceptance, cutover, rollback, and support.

EXTENSION CASE

> The strongest extension argument is not effort expended. It is a safer, faster, repeatable modernization method with one credible release, better system visibility, and a prioritized pipeline of next work.